TBfor 1.0.1

User Guide

Contents

1 Initials **5**
- 1.1 Compiling . . . 5
- 1.2 Allowed geometries . . . 5
- 1.3 Input files . . . 6
  - 1.3.1 The TB parameters file . . . 6
  - 1.3.2 The running mode file . . . 9

2 The main input file **11**
- 2.1 The parameters data block. . . . 11
- 2.2 The species data block . . . 13
- 2.3 The lattice data block . . . 14
- 2.4 The k-grid data block . . . 17
- 2.5 The k-plot data block . . . 17
- 2.6 The coordinates data block . . . 18

3 Running - Electronic Structure **21**
- 3.1 1D example - Nanotube . . . 22
- 3.2 2D example - Graphene . . . 24

4 Running - Hamiltonian generation **29**
- 4.1 Initial Electronic Structure run . . . 29
- 4.2 Needed files . . . 33
- 4.3 Generating the Hamiltonians . . . 34

Chapter 1

Initials

1.1 Compiling

- Start unpacking the tbfor tar file;

- Enter in the tbfor folder;

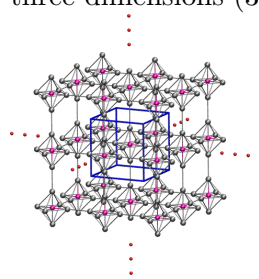
- Edit the makefile to specify *BLAS* and *LAPACK* and the mpi fortran compiler;

- Then just type make and (if everything is ok) that's it.

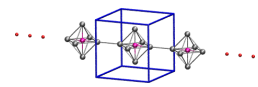
1.2 Allowed geometries

TBfor is able to work with structures which are periodic either in:

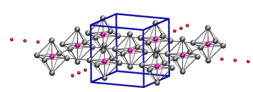
- three dimensions (3D);



- one dimension (1D);



- two dimensions (2D);



- or in no direction (0D).

1.3 Input files

We need three input files to calculate the electronic structure of our crystalline system:

- The "`parametros.ent`" file, containing the TB parameters;
- The "`run.info`" file, determining the running mode;
- The "`systemlabel.ent`" file (where "`systemlabel`" is a string you choose to identity your simulation) is the main input and contains the atomic structure and other parameters.

1.3.1 The TB parameters file

In the "`parametros.dat`" file we have the values for the site energies and the hopping integrals between 1st, 2nd and 3rd nearest neighbors for the different species involved in the calculation (all in eV). It contains, in this order:

- Site energy for species 1
- 1st neighbor hopping parameter between species 1 and 1

- 2nd neighbor hopping parameter between species 1 and 1
- 3rd neighbor hopping parameter between species 1 and 1
- 1st neighbor hopping parameter between species 1 and 2
- 2nd neighbor hopping parameter between species 1 and 2
- 3rd neighbor hopping parameter between species 1 and 2
- ...
- 1st neighbor hopping parameter between species 1 and N
- 2nd neighbor hopping parameter between species 1 and N
- 3rd neighbor hopping parameter between species 1 and N
- Site energy for species 2
- 1st neighbor hopping parameter between species 2 and 2
- 2nd neighbor hopping parameter between species 2 and 2
- 3rd neighbor hopping parameter between species 2 and 2
- 1st neighbor hopping parameter between species 2 and 3
- 2nd neighbor hopping parameter between species 2 and 3
- 3rd neighbor hopping parameter between species 2 and 3
- ...
- 1st neighbor hopping parameter between species 2 and N
- 2nd neighbor hopping parameter between species 2 and N
- 3rd neighbor hopping parameter between species 2 and N
- ... and so on ...

If overlap matrices are included in the calculation, the you have to have a "`parametros_s.dat`" file with the values for the overlap integrals between 1st, 2nd and 3rd nearest neighbors for the different species involved in the calculation. It contains, in this order:

- 1st neighbor overlap parameter between species 1 and 1
- 2nd neighbor overlap parameter between species 1 and 1
- 3rd neighbor overlap parameter between species 1 and 1
- 1st neighbor overlap parameter between species 1 and 2
- 2nd neighbor overlap parameter between species 1 and 2
- 3rd neighbor overlap parameter between species 1 and 2
- ...
- 1st neighbor overlap parameter between species 1 and N
- 2nd neighbor overlap parameter between species 1 and N
- 3rd neighbor overlap parameter between species 1 and N
- 1st neighbor overlap parameter between species 2 and 2
- 2nd neighbor overlap parameter between species 2 and 2
- 3rd neighbor overlap parameter between species 2 and 2
- 1st neighbor overlap parameter between species 2 and 3
- 2nd neighbor overlap parameter between species 2 and 3
- 3rd neighbor overlap parameter between species 2 and 3
- ...
- 1st neighbor overlap parameter between species 2 and N
- 2nd neighbor overlap parameter between species 2 and N
- 3rd neighbor overlap parameter between species 2 and N
- ... and so on ...

1.3.2 The running mode file

In the "`run.info`" we determine the running mode. It contains a single line with either:

- "`tb`" for an electronic structure calculation, or;
- "`hs`" to generate Hamiltonian and overlap files for a transport calculation to be used in either TRANSFOR or TRANSPLAYER.

In the following we describe the main input file.

Chapter 2

The main input file

The main input (systemlabel.ent) has the system label and a number of data blocks, each referring to a specific set of parameters. It has the following structure:

```
systemlabel

param_data
...
param_data

speci_data
...
speci_data

latti_data
...
latti_data

kgrid_data
...
kgrid_data

kplot_data
...
kplot_data

coord_data
...
coord_data
```

Where the first line of the file contains the string which will be in the name of all the output files. Let us now discuss each of the following data blocks separately.

2.1 The parameters data block.

The **parameters block** is opened and closed with the string `param_data`, so that you can input specific calculation options within it:

- `pul` - Determines the Pulay mixing parameter. Allowed values: positive real numbers (default = 0.1);

- `his` - Determines the number of previous densities to be considered in the Pulay mixing. Allowed values: positive integers (default = 1);

- `tol` - Specifies the tolerance for variations of the electronic occupation in the self consistency cycle. Allowed values: positive real numbers (default = 10^{-8});

- `ite` - Limit the number of iterations in the self-consistency cycle. Allowed values: positive integers (default: no limit);

- `dos` - Tells the program if the DOS, PDOS and LDOS should be calculated or not. If set to "true", an additional line should be included containing the broadening (in eV) for the gaussian functions, the limits of the energy range and the number of energy points. Allowed values: `.true.` or `.false.` (default `.false.`);

- `vec` - Tells the program if the eigenvectors should be written. Warning: .vec files are usually huge. Allowed values: `.true.` or `.false.` (default `.false.`);

- `efd` - Tells the program if an electric field should be applied or not. If set to "true", an additional line should be included containing the x, y and z components for the electric field vector (in units of V/Å). Allowed values: `.true.` or `.false.` (default `.false.`);

- `fix` - Tells the program if the spin should be constrained for specific atoms. If set to "true", the atoms to be constrained will be specified in **coordinates block**. Allowed values: `.true.` or `.false.` (default `.false.`);

- `ove` - Tells the program if overlap matrices should be included or not. Allowed values: `.true.` or `.false.` (default `.false.`);

- `tcl` - Tells the program if the a tcl file with the spin plot should be written or not. Allowed values: `.true.` or `.false.` (default `.true.`).

This Block is optional, but we recommend to include it in accordance with what you want to calculate. See the example below:

```
param_data
 pul   1.0D0
 his   5
 tol   1.0D-7
 dos   .true.
 0.08 -1.0 1.0 100
 vec   .false.
 efd   .true.
 0.01  0.00  0.00
 fix   .true.
 ove   .false.
 param_data
```

2.2 The species data block

The **species block** is opened and closed with the string "`speci_data`". It contains information about the species used in the calculation. Inside it we include:

1. One line with the number of different species;
2. Then one line for each one, with an ID number, number of neighbors, minimum and maximum distance to 1st neighbors, minimum and maximum distance to 3rd neighbors (in Å) and the U value for it.

See the example below:

```
speci_data
 2
 1  12  1.3  1.94  2.65  3.00  2.944D0
 2  12  1.3  1.94  2.65  3.00  2.944D0
speci_data
```

In this example we have that:

- The distance between first-nearest neighbors lies in the range [1.3 Å, 1.94 Å].
- The distance between second-nearest neighbors lies in the range]1.94 Å, 2.65 Å].
- The distance between third-nearest neighbors lies in the range]2.65 Å, 3.0 Å].

This block is optional (even though we advise you to include it properly). If not present, TBFOR will consider the following as default:

```
speci_data
 1
 1  12  1.0D0  2.0D0  2.7D0  3.40D0  0.0D0
speci_data
```

2.3 The lattice data block

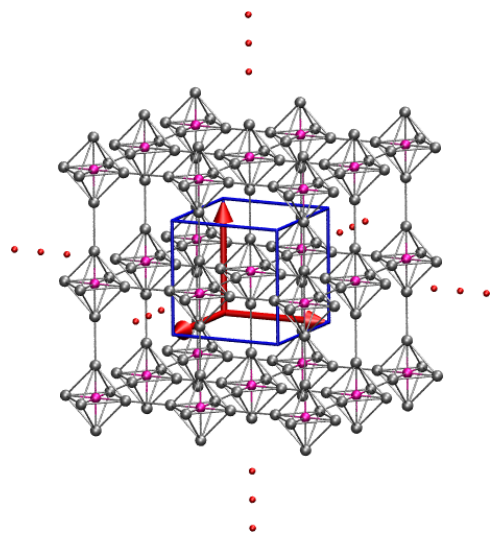
The **lattice block** is opened and closed with the string "`latti_data`". It contains information about the vector defining the box used in the calculation, namely:

$$
\begin{aligned}
\mathbf{a}_1 &= a_{1x}\hat{i} + a_{1y}\hat{j} + a_{1z}\hat{k} \\
\mathbf{a}_2 &= a_{2x}\hat{i} + a_{2y}\hat{j} + a_{2z}\hat{k} \\
\mathbf{a}_3 &= a_{3x}\hat{i} + a_{3y}\hat{j} + a_{3z}\hat{k}
\end{aligned}
$$

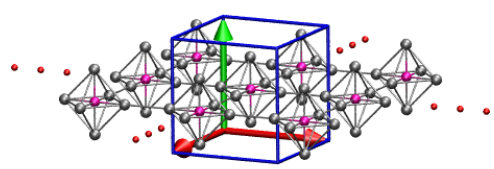
Inside it we include three lines, each with the coordinates (in Å) of a lattice vector. See the example below:

```
latti_data
  2.50   0.00   0.00
  0.00   3.00   0.00
  0.00   0.00   2.50
latti_data
```

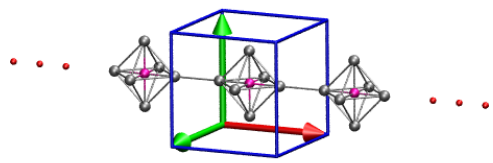
For a 3D system we just pick the lattice vectors according to the box in our 3D system, as depicted below:



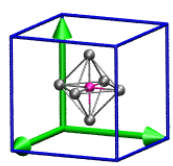
For a 2D system we pick the two lattice vectors (red) and for the remaining one we choose a very long vector (green) out of their plane. This is to avoid the stacking layers to interact with each other.



For a 1D system we pick the single lattice vector (red) and two very long vectors (green) whose plane does not contain the single lattice vector (the plane can be orthogonal, for example). This is to avoid the parallel images of the system to interact with each other.



For a 0D system we just pick three vectors, not in the same plane, long enough to make a box such that the system do not interact with its periodic images. We can pick the vectors to be orthogonal, for instance.



This block is optional. If not present, the default values are:

$$\begin{aligned}\mathbf{a}_1 &= (1.0\mu m)\mathbf{i}\\ \mathbf{a}_2 &= (1.0\mu m)\mathbf{j}\\ \mathbf{a}_3 &= (1.0\mu m)\mathbf{k}.\end{aligned}$$

2.4 The k-grid data block

The **k-grid block** is opened and closed with the string "`kgrid_data`". It contains information about the sampling to be used for integrals over the reciprocal space aiming the calculation of the site occupancies in the self consistency cycle. For the grid, you can choose between two options:

1. "`eqspace`": for a set of equally spaced points;
2. "`wpoints`": when you enter explicitly the points to be used and their corresponding weights.

For the first case, you start with a line containing the string "`eqspace`". Then include a line with three integer representing the number of k points along each basis vector from the reciprocal lattice. See example below:

```
kgrid_data
eqspace
1 4 4
kgrid_data
```

For the second case, you start with a line containing the string "`wpoints`". Then include a line with the number of points to be used. So you enter a line for each point with its weight and the coefficients for the linear combination of the reciprocal lattice vectors which give the corresponding k vector. See example below:

```
wpoints
4
0.25  0.5 0.4 0.4
0.25  0.4 0.5 0.4
0.25  0.4 0.4 0.5
0.50  0.0 0.0 0.0
```

This block is optional. If absent, the default scheme is "`eqspace`" with 1 1 1.

2.5 The k-plot data block

The **k-plot block** is opened and closed with the string "`kplot_data`". It determines how the bands will be calculated along the Brillouin zone. For the plot, you can choose between two options:

1. "`lines`": Plotting along a polygonal line over the reciprocal space;
2. "`bzone`": Plotting on a grid over the Brillouin zone.

For the first case, you start with a line containing the string "`lines`". Then:

1. A line with the number of corners along this path and the number of k points from the first corner to the second (the number of points along the other sides of the polygonal path will be proportional to the length of the side in the k-space);
2. Then one line for each corner containing its position (coefficients of a linear combination of the basis vectors for the reciprocal space).

See example below:

```
kplot_data
lines
 4 100
 0.667D0  0.333D0  0.000D0
 0.000D0  0.000D0  0.000D0
 0.500D0  0.500D0  0.000D0
 0.667D0  0.333D0  0.000D0
kplot_data
```

For the second case, you start with a line containing the string "`bzone`". Then a line with three integer representing the number of k points along each basis vector from the reciprocal lattice. See example below:

```
kplot_data
bzone
 100  100  1
kplot_data
```

This block is optional. If absent, the default scheme is "`lines`" with two corners (both equal to the Γ point).

2.6 The coordinates data block

The **coordinates block** is opened and closed with the string "`coord_data`". It contains information about the atomic positions for the structure. It contains:

1. One line with the number of atoms and the number of electrons;
2. Then one line for each atom with the species ID, the x, y and z coordinates for the atom and the fix-spin flag (if the "`fix`" option is set to true in the "`param_data`" block):

The fix-spin flag is 0 (zero) if you want the spin-up and -down occupations to relax during the self-consistency cycle, or 1 if you want to fix it. You do not need to include the fix-spin flag if the "`fix`" option is set to "`.false.`" in the parameters data block. See example below (with "`fix`" set to "`.true.`"):

```
coord_data
10  10
2           0.0000000  0.0000000    0.7100000     1
2           0.0000000  0.0000000    2.1300000     0
1           1.2297561  0.0000000    0.0000000     0
1           1.2297561  0.0000000    2.8400000     0
1           2.4595121  0.0000000    0.7100000     0
1           2.4595121  0.0000000    2.1300000     0
1           3.6892682  0.0000000    0.0000000     0
1           3.6892682  0.0000000    2.8400000     0
2           4.9190242  0.0000000    0.7100000     0
2           4.9190242  0.0000000    2.1300000     0
coord_data
```

This block is not optional. IT MUST BE INCLUDED!

Chapter 3

Running - Electronic Structure

Once you have the executable and the two input files in the same folder, you do the following

- Make a copy (do not just rename, but copy) of the "`systemlabel.ent`" file and name the copy as "`input1.ent`" (you can use any integer instead of 1 or either none - "`input.ent`");
- Run the executable.
- Note that at the end, the copy "`input1.ent`" will be deleted.

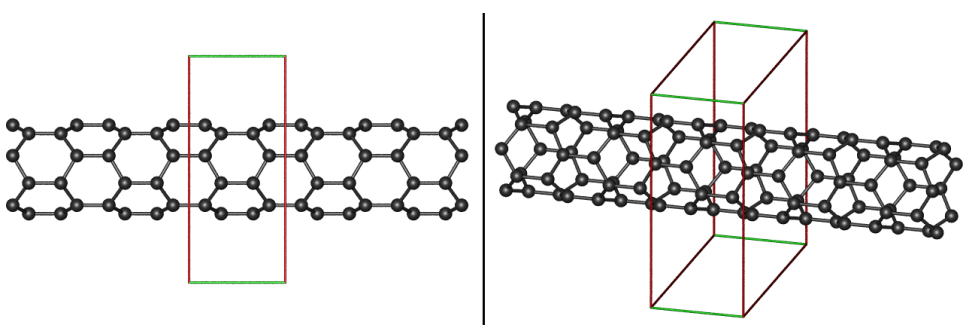
The output files are:

- "`systemlabel.mag`": contains the self-consistent occupations for spin-UP and -DOWN for the corresponding basis orbitals;
- "`systemlabel.gap`": contains the electronic band gap around the Fermi energy and the Fermi energy value;
- "`systemlabel-bands.dat`": contains the electronic band structure;
- "`systemlabel.vec`": contains the eigenvectors (to be used in used in STM simulations);
- "`systemlabel.tcl`": contains the info for plotting spin-polarization in `vmd`;
- "`systemlabel-energy.inf`": contains the band energy;

- "`systemlabel-dos-tot.dat`": contains the total DOS;
- "`systemlabel-dos-up.dat`": contains the spin-up DOS;
- "`systemlabel-dos-dw.dat`": contains the spin-dw DOS;
- "`systemlabel-pdos-tot.dat`": contains the total PDOS;
- "`systemlabel-pdos-up.dat`": contains the spin-up PDOS;
- "`systemlabel-pdos-dw.dat`": contains the spin-dw PDOS;
- "`systemlabel-ldos-tot.dat`": contains the total LDOS;
- "`systemlabel-ldos-up.dat`": contains the spin-up LDOS;
- "`systemlabel-ldos-dw.dat`": contains the spin-dw LDOS;

3.1 1D example - Nanotube

Let us consider a $(5, 0)$ nanotube. The supercell will be given by the box drawn below.



Note that we choose the unit cell's length (green length) to be exactly the length of a nanotube unit cell ($3 \cdot a_{CC}$). On the other hand, the two other lengths (red lengths) are chosen very long. Here is the input file:

```
tube50

param_data
 pul   1.0D0
 his   5
 dos   .false.
 vec   .false.
 efd   .false.
 fix   .false.
param_data

speci_data
 1
 1  12  1.3  1.94  2.65  3.00   2.944
```

```
speci_data

latti_data
 1000.000      0.000   0.000
    0.000  1000.000    0.000
    0.000     0.000    4.260
latti_data

kgrid_data
eqspace
1  1  10
kgrid_data

kplot_data
lines
  2 100
 0.0D0 0.0D0 0.0D0
 0.0D0 0.0D0 0.5D0
kplot_data

coord_data
20  20
1        1.957218000      0.000000000      1.420000000
1        1.583422000     -1.150424000     -0.710000000
1        0.604813000     -1.861425000      1.420000000
1       -0.604813000     -1.861425000     -0.710000000
1       -1.583422000     -1.150424000      1.420000000
1       -1.957218000      0.000000000     -0.710000000
1       -1.583422000      1.150424000      1.420000000
1       -0.604813000      1.861425000     -0.710000000
1        0.604813000      1.861425000      1.420000000
1        1.583422000      1.150424000     -0.710000000
1        1.583422000     -1.150424000      0.710000000
1        0.604813000     -1.861425000     -1.420000000
1       -0.604813000     -1.861425000      0.710000000
1       -1.583422000     -1.150424000     -1.420000000
1       -1.957218000      0.000000000      0.710000000
1       -1.583422000      1.150424000     -1.420000000
1       -0.604813000      1.861425000      0.710000000
1        0.604813000      1.861425000     -1.420000000
1        1.583422000      1.150424000      0.710000000
1        1.957218000      0.000000000     -1.420000000
coord_data
```

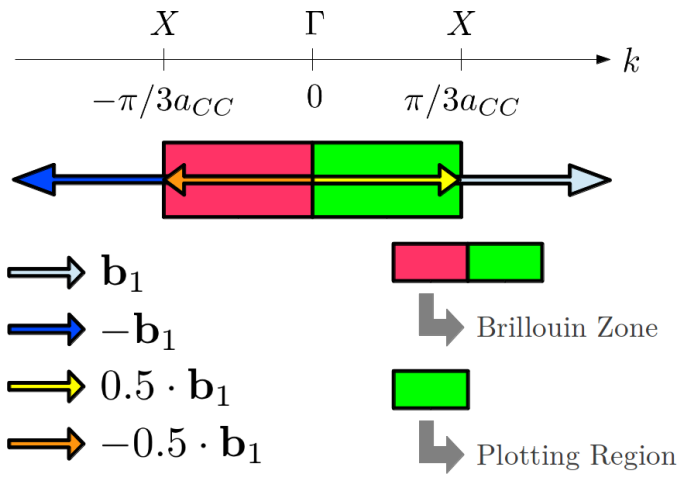
The supercell vectors are given by:

$$\begin{aligned}\mathbf{a}_1 &= L\hat{i}\\ \mathbf{a}_2 &= L\hat{j}\\ \mathbf{a}_3 &= 3a_{CC}\hat{k}\end{aligned}$$

And the reciprocal vectors are:

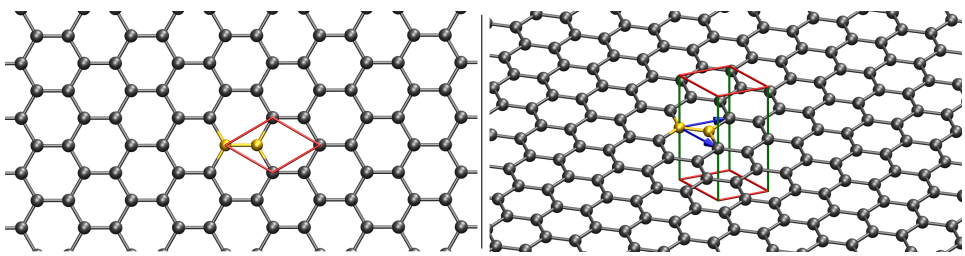
$$\begin{aligned}\mathbf{b}_1 &= \frac{2\pi}{L}\hat{i}\\ \mathbf{b}_2 &= \frac{2\pi}{L}\hat{j}\\ \mathbf{b}_3 &= \frac{2\pi}{3a_{CC}}\hat{k}\end{aligned}$$

from which the third is the basis vector for the one-dimensional reciprocal space. The 1D Brillouin zone extends from $-0.5 \cdot \mathbf{b}_3$ to $0.5 \cdot \mathbf{b}_3$, but we plot the bands only from $\mathbf{0}$ to $0.5 \cdot \mathbf{b}_3$ due to time reversal symmetry.



3.2 2D example - Graphene

Let us now consider a graphene sheet. The supercell will be given by the box drawn below.

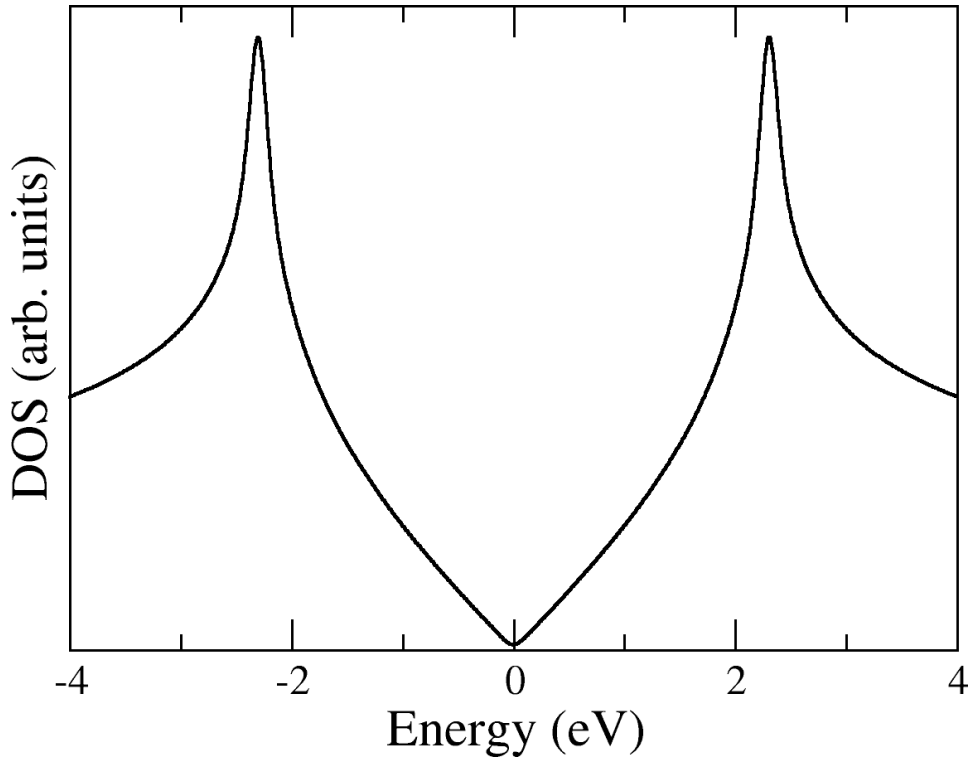


Note that we choose the basis plane of the supercell (red lengths) to be the graphene unit cell unit cell. On the other hand, the other length (green) is chosen very long to avoid interactions between the stacking layers. Here is the input file:

```
graphene_bz_plot

param_data
 pul   1.0D0
 his   5
 dos  .true.
 0.08  -4.0  4.0  801
 vec  .false.
 efd  .false.
 fix  .false.
param_data

speci_data
 1
 1  12  1.3  1.94  2.65  3.00  2.944
speci_data

latti_data
  2.13   1.23   0.00
```

```
  2.13  -1.23   0.00
  0.00   0.00   8.00
latti_data

kgrid_data
eqspace
 12  12  1
kgrid_data

kplot_data
bzone
 400  400  1
kplot_data

coord_data
2  2
1   0.000   0.000    0.000
1   1.420   0.000    0.000
coord_data
```

Here are the resulting bands and DOS:



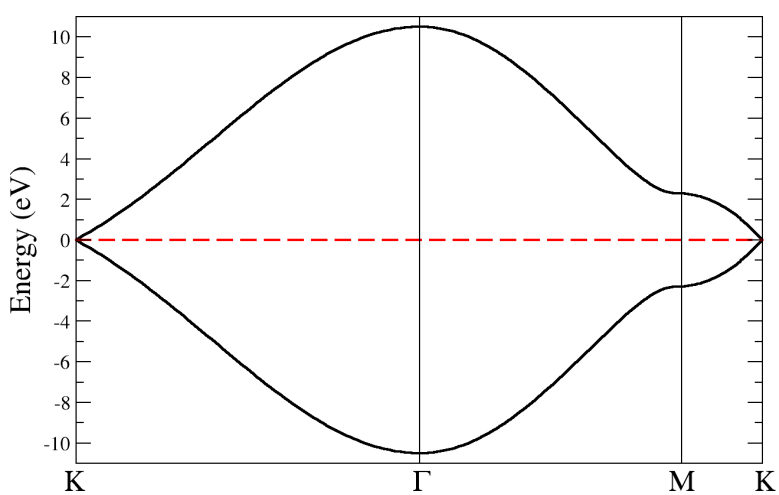
Now using the lines approach to plot the bands:

```
graphene_bz_plot

param_data
 pul   1.0D0
 his   5
 dos  .true.
 0.08  -4.0  4.0  801
 vec  .false.
 efd  .false.
 fix  .false.
param_data

speci_data
 1
 1  12  1.3  1.94  2.65  3.00  2.944
speci_data

latti_data
  2.13   1.23   0.00
  2.13  -1.23   0.00
  0.00   0.00   8.00
latti_data

kgrid_data
eqspace
 12  12  1
kgrid_data

kplot_data
lines
 4 100
 0.667D0 0.333D0 0.000D0
 0.000D0 0.000D0 0.000D0
```

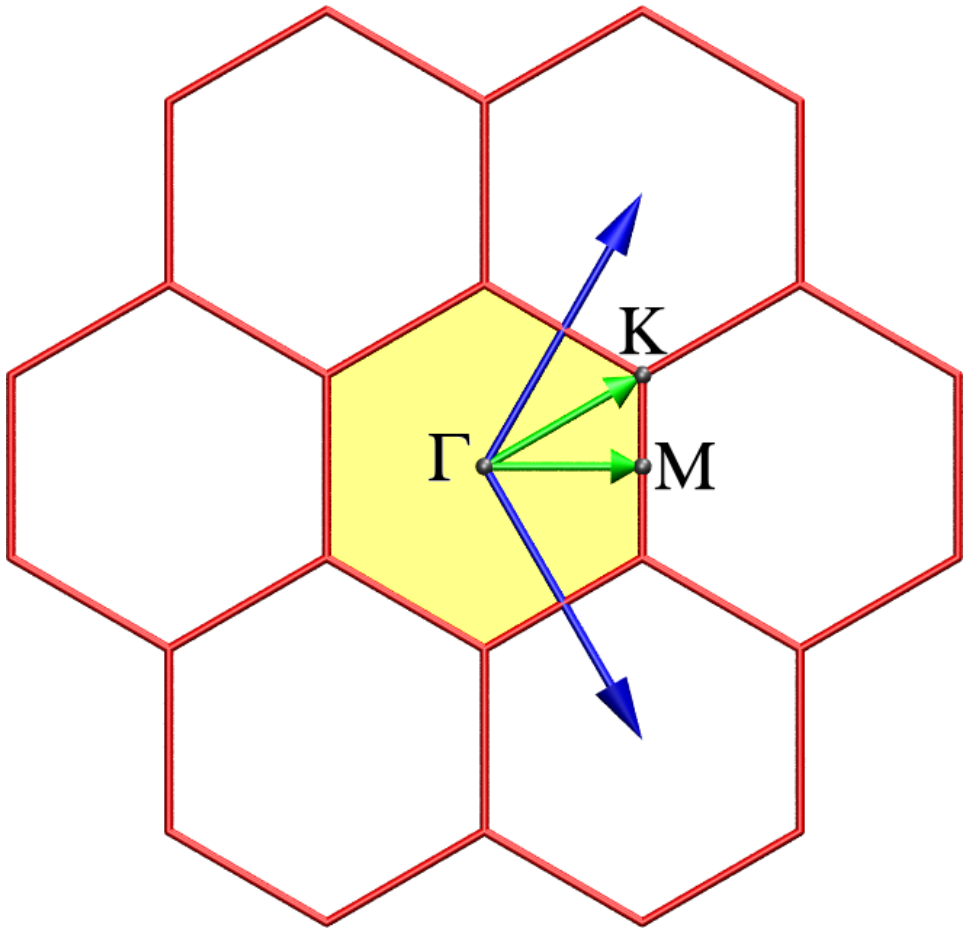
```
 0.500D0 0.500D0 0.000D0
 0.667D0 0.333D0 0.000D0
kplot_data

coord_data
2  2
1   0.000   0.000    0.000
1   1.420   0.000    0.000
coord_data
```

Remark: do not calculate DOS when using this scheme, unless you are simulating a 1D or 0D system.

Here are the resulting bands



See below the Brillouin zone for graphene (in yellow).



The plot starts at the K point, then goes to Γ, M and back to K:

$$\begin{aligned}
\mathbf{k}_K &= (2\mathbf{b}_1 + \mathbf{b}_2)/3 \\
\mathbf{k}_\Gamma &= \mathbf{0} \\
\mathbf{k}_M &= (\mathbf{b}_1 + \mathbf{b}_2)/2
\end{aligned}$$

Chapter 4

Running - Hamiltonian generation

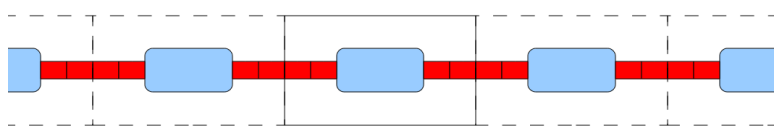
For the Hamiltonian Generation you have to have the same inputs from an Electronic Structure calculation (with the hs option in the running mode file) plus the outputs from a previous Electronic Structure calculation. You do not need to rename the systemlabel.ent file, it will not be deleted in this running mode. In order to generate the Hamiltonians, simply execute TBfor and it will ask you for:

- The name of the main input file;
- The number of terminals
- The number of atoms in the unit cell for terminal 1, 2, 3, ...
- If you want to write the total Hamiltonians;
- If you want to write the spin-up Hamiltonians;
- If you want to write the spin-down Hamiltonians;
- Which format you want to use to write the Hamiltonians.

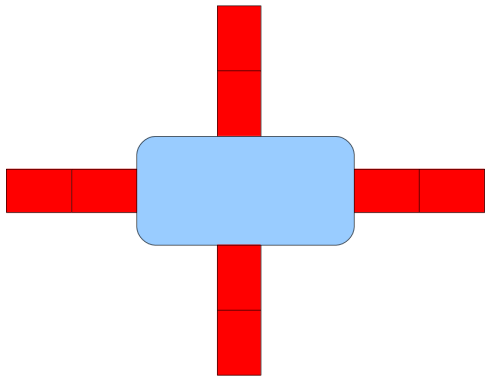
4.1 Initial Electronic Structure run

You have to use TBfor to simulate a *periodic boundary conditions* version of your *Central Conductor + Terminals* (two cells from each lead) system. Let us suppose a two terminal system as depicted below:

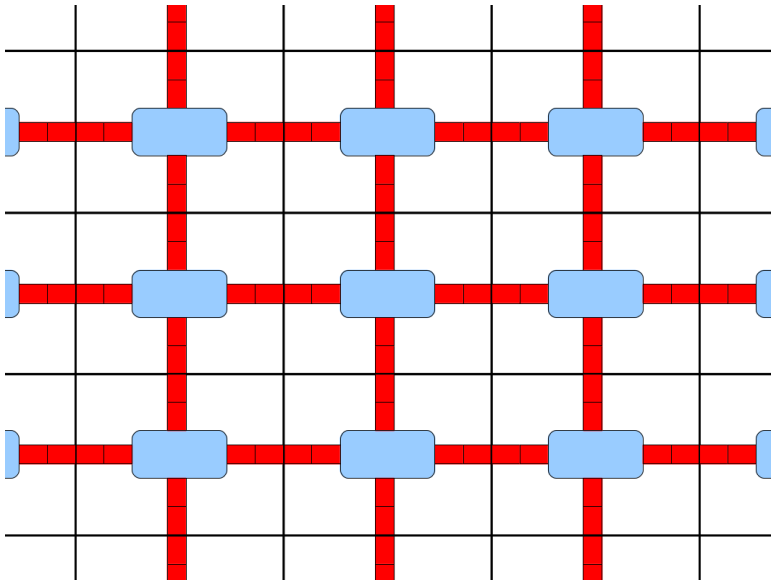
Then you make a tbfor run by simulating the (Conductor + two cells from each terminal) system in periodic boundary conditions like this:



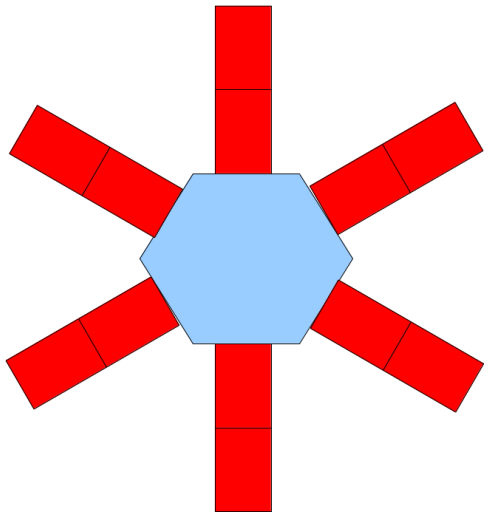
Let us suppose a four terminal system with 90° angles between the terminals:



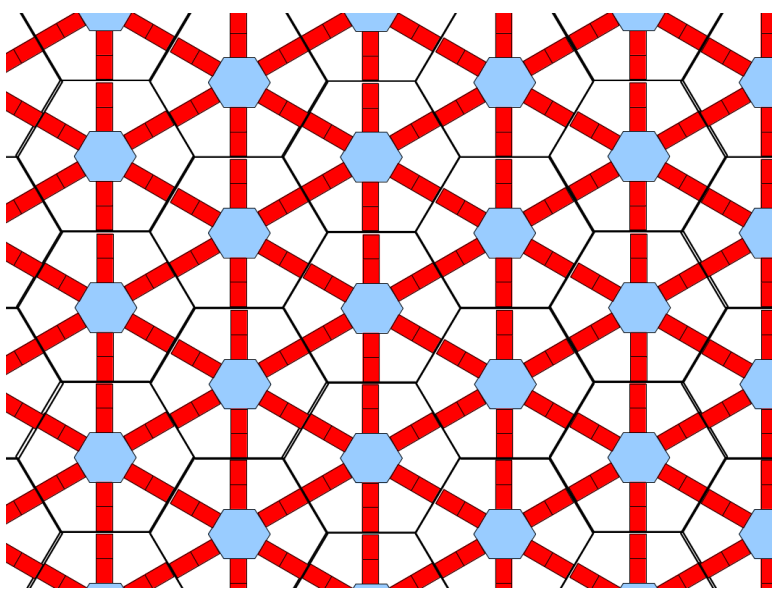
So its *periodic boundary conditions* version will look like this:



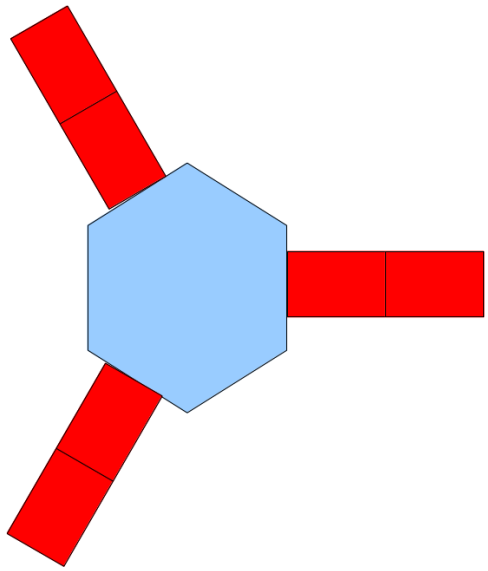
And finally a six terminal system with 60° angles between the terminals:



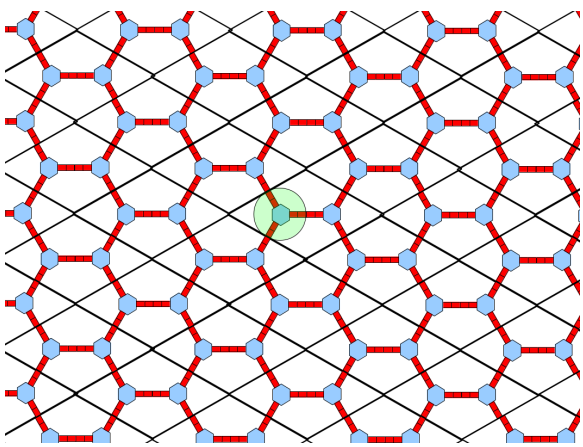
Here is the *periodic boundary conditions* version:



For last a three terminal system with 120° angles between the terminals:



Here is the *periodic boundary conditions* version:



and you take only one half of the system for the subsequent work.

4.2 Needed files

You will then use the following output files from tbfor:

- "`systemlabel.mag`".
- "`systemlabel.gap`".

as well as the inputs

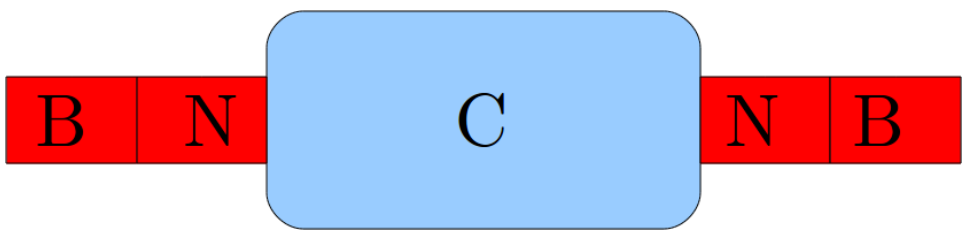
- "`systemlabel.xyz`".
- "`systemlabel.ent`".

The "`systemlabel.ent`", "`systemlabel.mag`" and "`systemlabel.gap`" files are the same as those for the electronic structure run. They may need to be edited depending on the system's setup (see the three terminals case).

The "`systemlabel.xyz`" contains the number of atoms in the first line, one empty line and then one line for each atom, containing:

- Species label (C, N or B);
- x y z coordinates.

where we use C for the conductor atoms, N for the terminal atoms in the cell closer to the conductor and B for the terminal cells in the cell not directly attached to the electrodes.



4.3 Generating the Hamiltonians

Just execute the program. Then it will ask for

- The "`systemlabel.ent`" file;
- The number of terminals;
- The number of atoms in each terminal cell;
- If the total (spin-up+down) Hamiltonian and overlap should be written;
- If the spin-up Hamiltonian and overlap should be written;
- If the spin-down Hamiltonian and overlap should be written;
- The files format (ext, pla or spa).

TBfor will also write an "`input.inp`" file for the TRANSfor run. You can edit this file according to what you need.